

Context Window, Effort Levels & Fast Mode

1M Context: v2.1.71 | /effort: v2.1.66 | Fast Mode: v2.1.50 | Latest: v2.1.76

What They Are

Three related features that control how Claude Code thinks:

- **1M Token Context Window** — 5x larger context, now default for Opus 4.6
- **/effort Command** — Dial reasoning depth up or down per session
- **Fast Mode** — Same model, faster output, lower latency

Plus complementary session management: - **/context Command** — See what's consuming your context and get optimization tips - **/color Command** — Visually distinguish parallel sessions - **Session Naming** — -n / --name flag for labeling sessions

1M Token Context Window

What Changed

Previously, Opus had a 200K token context window. Now **1M tokens** is the default for Opus 4.6 on Max, Team, and Enterprise plans.

What 1M Tokens Means in Practice

Content	Approximate tokens
200K context (old)	~150K lines of code
1M context (new)	~750K lines of code
Average source file	~500-2000 tokens
Full medium codebase	Fits in single session

When It Matters

- **Large refactors** — keep entire module in context while restructuring
- **Cross-file analysis** — read 50+ files without compaction
- **Long sessions** — work for hours without losing early context
- **Agent teams** — each agent gets full 1M window

Context Compaction Still Happens

Even with 1M tokens, compaction triggers when context fills. The difference is it takes much longer to fill, and compaction preserves more information in the larger window.

/effort Command

What It Does

Controls how much reasoning Claude applies to each response. Higher effort = deeper thinking = slower but more thorough. Lower effort = faster but more surface-level.

Usage

```
/effort high      # Deep reasoning (extended thinking)
/effort standard  # Normal reasoning
/effort low       # Quick responses, minimal reasoning
```

When to Use Each

Level	Best for	Trade-off
high	Complex refactors, architecture decisions, debugging	Slower, uses more tokens
standard	Normal development tasks	Balanced (default)
low	Simple edits, file reads, quick questions	Fast but less thorough

In Agent/Skill Frontmatter

```
---
name: deep-analysis
model: opus
effort: high
---
```

Setting alwaysThinkingEnabled

For persistent deep thinking across all sessions:

```
{
  "alwaysThinkingEnabled": true
}
```

Fast Mode

What It Does

/fast toggles faster output from the **same model** (Opus 4.6). It's not a model downgrade — it optimizes for lower latency at the cost of some reasoning depth.

Usage

```
/fast          # Toggle fast mode on/off
```

When It Appeared

Fast mode was available since v2.1.50 for Opus 4.6 with full 1M context support.

Fast Mode vs /effort

Feature	What changes	Model changes?
/fast	Output speed/latency	No (same model)

Feature	What changes	Model changes?
/effort	Reasoning depth	No (same model)

You can combine them: `/fast on + /effort low` for maximum speed on simple tasks.

/context Command

What It Does

Shows what's consuming your context window and suggests optimizations:

`/context`

Output includes: - Current token usage (e.g., "342K / 1M tokens used") - Breakdown by category (conversation, tool results, system prompts, memory) - Actionable suggestions (e.g., "Consider compacting — 3 large tool results could be freed") - Warning thresholds (yellow at 70%, red at 90%)

When to Use

- Before starting a large task (check available space)
 - When responses feel degraded (context may be full)
 - After reading many files (see the impact)
-

/color Command

What It Does

Assigns a color to the current session for visual distinction when running multiple sessions:

```
/color          # Assign a random color
/color blue     # Specific color
/color reset    # Remove color
```

When It Matters

Running 3+ sessions in separate terminals? Colors help you instantly identify which session is which — the prompt/border changes color.

Session Naming (-n / --name)

What It Does

Label sessions at startup for easier identification:

```
claude -n "auth refactor"
claude --name "payment-v2"
```

Combined with /color

```
# Terminal 1
claude -n "backend" && /color blue

# Terminal 2
claude -n "frontend" && /color green

# Terminal 3
claude -n "tests" && /color yellow
```

In Session Resume

Named sessions are easier to find in the session picker (claude -r):

Recent sessions:

1. auth refactor (2h ago) – /Users/me/project
2. payment-v2 (3h ago) – /Users/me/project
3. tests (5h ago) – /Users/me/project

Previously you could only name sessions with /rename after starting — now you can name them upfront.

Real-World Workflows

Workflow 1: Large Codebase Exploration

```
claude -n "codebase-audit"
> /effort high
> /context
# Shows: 0 / 1M tokens used
> Read the entire src/ directory and give me an architecture overview
# With 1M context, can read 500+ files without compaction
```

Workflow 2: Quick Fixes at Speed

```
claude -n "quick-fixes"
> /fast
> /effort low
> fix the typo in README.md
> add the missing import in utils.ts
> rename getUserData to fetchUserData
# Each response in under 2 seconds
```

Workflow 3: Parallel Session Management

```
# Terminal 1
claude -n "feature-auth" -w auth-branch
> /color blue

# Terminal 2
claude -n "bugfix-login" -w login-fix
> /color red

# Terminal 3
```

```

claude -n "tests"
> /color green
> /loop 10m check if the CI is green for auth-branch and login-fix

```

How They Interact With Other Features

Feature	Interaction
Agent Teams	Each teammate gets full 1M context window
Worktrees	Session naming helps identify which worktree is which
Session Resume	Named sessions easier to find. Context preserved on resume
Remote Control	Named sessions discoverable when connecting remotely
Auto-Memory	Memory files consume minimal context (~1-2K tokens)
/loop	Loops work in fast mode — quick periodic checks
Skills	Skills can set effort level in frontmatter

Gotchas & Tips

1. **1M context ≠ unlimited** — it's 5x bigger but still finite. Monitor with `/context`.
2. **Extended thinking tokens** — thinking tokens count separately from context. `/effort high` uses thinking tokens, not context tokens.
3. **Fast mode is per-session** — doesn't persist across session resume.
4. **/color is visual only** — it doesn't affect behavior, just terminal display.
5. **Session naming is for display** — the `-n` flag sets a display name, not a unique ID. You can have multiple sessions with the same name.
6. **Plan availability** — 1M context requires Max, Team, or Enterprise plan. Pro plan stays at 200K.

Version History

Version	Date	Change
v2.1.50	Feb 2026	Fast mode with 1M context for Opus 4.6
v2.1.57	Mar 2026	<code>/context</code> command
v2.1.60	Mar 2026	<code>/color</code> command available to all users
v2.1.66	Mar 2026	<code>/effort</code> command for session-level effort control
v2.1.71	Mar 2026	1M context default for Opus 4.6 (Max/Team/Enterprise)
v2.1.74	Mar 2026	Model fallback human-friendly names
v2.1.76	Mar 2026	Session naming (<code>-n/--name</code>), enterprise feedbackSurveyRate